DATABASE PORTFOLIO: PROJECT 2

PETROSTREAM OPERATIONS & PRODUCTION SYSTEMS

1. The Scene & Business Objective

The Scenario: PetroStream runs upstream oil and gas assets across major extraction fields, splitting assets between onshore fields and complex offshore deepwater rigs.

The Operational Challenges:

- **Asset Cost Disparity:** Offshore exploration requires intense operational budgets. Production output metrics must be decoupled by field type to evaluate financial efficiency.
- **Vendor Maintenance Variances:** Repair costs fluctuate drastically. Management needs an automated architecture to capture anomalies where individual maintenance logs exceed historical national averages.
- **Data Volume Scaling:** Daily extraction metrics log tens of thousands of performance values continuously, causing standard point lookups to experience performance degradation over time.

Objective: To deploy an optimized monitoring schema that handles metric isolation via views, dynamic subqueries, advanced procedures, and rapid index seek patterns.

2. Simplified Tasks, Queries, and Technical Outputs

Task 1: The Offshore Production Audit

Goal: Calculate the exact average daily barrel performance specifically for high-cost offshore wells to ensure operational targets are met.

Query Implemented:

```
WITH AverageDailyOilProduction_CTE AS (
    SELECT Wells.WellID, Region, Well_Type, AVG(Barrels_Oil) AS AVG_OIL_PRD
    FROM Production
    JOIN Wells ON Production.WellID = Wells.WellID
    WHERE Well_Type = 'Offshore'
    GROUP BY Wells.WellID, Region, Well_Type
)
SELECT * FROM AverageDailyOilProduction_CTE
ORDER BY AVG_OIL_PRD DESC;
```

Output Dataset:

WellID	Region	Well_Type	AVG_OIL_PRD
W-03	Bonga Field	Offshore	8000
W-01	Niger Delta	Offshore	3266

Task 2: High-Cost Maintenance Leak Detection

Goal: Dynamically cross-reference all historical maintenance repair invoices and extract rows that cost more than the company baseline average of $28,500.00.

Query Implemented:

```
CREATE VIEW v_RepairCostAboveAVG AS
SELECT
    Maintenance.WellID,
    Region,
    IssueDate,
    RepairCost_USD
FROM Maintenance
JOIN Wells ON Maintenance.WellID = Wells.WellID;
GO

SELECT *
FROM v_RepairCostAboveAVG
WHERE RepairCost_USD > (SELECT AVG(RepairCost_USD) FROM v_RepairCostAboveAVG)
ORDER BY RepairCost_USD DESC;
```

Output Dataset:

WellID	Region	IssueDate	RepairCost_USD
W-01	Niger Delta	2026-05-03	45000.00

Task 3: Regional Production Summary Engine

Goal: Provide data transparency for global operations leaders by constructing an on-demand engine that aggregates total volumes by region while natively preventing execution breaks if variables are omitted.

Query Implemented:

```
CREATE PROCEDURE sp_RegionalOilProductionSUM
    @TargetRegion VARCHAR(50) = NULL
AS
SELECT
    Wells.WellID,
    Region,
    SUM(Barrels_Oil) AS SUM_Oil_Production
FROM Wells
JOIN Production ON Wells.WellID = Production.WellID
WHERE (Region = @TargetRegion OR @TargetRegion IS NULL)
GROUP BY Wells.WellID, Region
ORDER BY SUM_Oil_Production DESC;
GO
```

Testing the Engine:

```
EXEC sp_RegionalOilProductionSUM @TargetRegion = 'Niger Delta';
```

Output Dataset:

WellID	Region	SUM_Oil_Production
W-01	Niger Delta	9800
W-02	Niger Delta	6150

Task 4: High-Velocity Production Indexing

Goal: Eradicate high-overhead "Table Scans" inside active transaction tables by generating dedicated physical address pointers for search variables.

Query Implemented:

```
CREATE INDEX IX_Production_Well_Date
ON Production (WellID, LogDate);
```

3. Conclusion & Core Competencies Shown

This project confirms enterprise backend implementation capabilities including:

- **Defensive Stored Procedures:** Using argument-fallback scripts (`OR @Variable IS NULL`) to keep executive application layers stable.
- **Aggregations and Overrides:** Utilizing CTE abstractions and views to calculate distinct mathematical values without creating query bloat.

- **Proactive Database Tuning:** Applying composite keys onto highly volatile time-series production records to ensure index seeks run smoothly.